GUÍA DOCENTE · SESIÓN 3 · UA1

Entorno en el laboratorio y mi primer script

Instalación del entorno en el laboratorio; la etiqueta script, los archivos .js, la sintaxis básica y la consola; práctica guiada con errores en vivo y práctica autónoma en clase

Módulo	CSTI12004 Programación web en el entorno cliente
Fecha	jueves 12 de noviembre de 2026 · presencial
Unidad de aprendizaje	UA1 · Elaboración de componentes de software
Capacidad	1.1 Elaborar componentes de software mediante el lenguaje JavaScript, según los estándares de desarrollo vigentes.
Saberes	Entorno de trabajo en el laboratorio: VS Code, extensiones, Git y clonación del repositorio · Uso de la etiqueta <script> y archivos .js · Sintaxis básica: comentarios, punto y coma, secuencias de escape y elementos del código · Consola del navegador y lectura de errores
Ser	Autonomía · Resolver las situaciones derivadas del trabajo de modo independiente (valor: responsabilidad).
Convivir	Trabajo en equipo · Acata los lineamientos y acuerdos establecidos por el grupo para el logro de objetivos comunes (valor: bien común).
Duración	4 horas, de 1:30 a 5:30 p. m., presencial en el laboratorio (incluye un receso)
Persona facilitadora	Giovanni Antonio Coto Calderón

Cómo usar esta guía

La **presentación** (7 diapositivas) contiene los temas nuevos; sus notas del orador están copiadas en la sección 3 con el código y la explicación línea a línea.

Las salidas de consola son **ejecuciones reales** en Chromium, servidas como lo hace Live Server. Los programas están en **programas_js/** (exposición, práctica guiada, errores y soluciones).

La sesión empieza **repitiendo la instalación** en las computadoras del laboratorio con la sección 8 de **S02_Guia_instalacion_entorno_VSCode**. La **práctica autónoma** se hace **en clase** (45 minutos).

1. Cronograma

Minutos	Momento	Recurso
0–10	Repaso	Sección 2.1
10–70	Instalación y configuración en el laboratorio	S02_Guia_instalacion_entorno_VSCode, sección 8 · sección 2.2
70–115	Exposición	Diapositivas 2–7 · sección 3

Minutos	Momento	Recurso
115–125	Receso	—
125–175	Práctica guiada	VS Code · sección 4
175–220	Práctica autónoma	S03_Practica_autonoma_Estudiantes · sección 5
220–230	Síntesis	Sección 6
230–235	Próxima sesión	Sección 7
235–240	Cierre: push y cierre de sesión de GitHub	Sección 8

2. Inicio

2.1 Repaso (10 min)

Pregunta	Respuesta esperada
¿Qué diferencia hay entre un error de sintaxis y uno de semántica?	El de sintaxis impide ejecutar el archivo; el de semántica se ejecuta, pero da un resultado incorrecto.
En la práctica de pruebas.js, ¿por qué se comentaba la línea con el error de sintaxis?	Para que el resto del archivo pudiera ejecutarse y encontrar los demás errores.

Pregunta de la sesión anterior: ¿Dónde conviene escribir el código JavaScript: dentro del HTML o en un archivo aparte? ¿Por qué? (Respuesta esperada: en un archivo .js aparte, para separar el comportamiento de la estructura y poder reutilizarlo; es el tema de hoy).

2.2 Instalación y configuración en el laboratorio (60 min)

Se repite la Parte 1 de la guía de instalación en la computadora del laboratorio que cada estudiante usará en las sesiones presenciales. Siga la **sección 8** de **S02_Guia_instalacion_entorno_VSCode**: el repositorio **no se crea de nuevo**, se **clona**.

Bloque	Qué hacer	Min
Revisión del equipo	Navegador y F12; VS Code instalado o User Installer; git --version.	5
Extensiones y configuración	Live Server, Prettier, Error Lens; settings.json del curso.	15
Identidad de Git	Con --global si cada estudiante tiene usuario de Windows propio; si no, git config user.name / user.email dentro del repositorio.	10
Clonar el repositorio	mkdir -p /c/INA · cd /c/INA · git clone URL · cd … · code .	15
Prueba	UA1/S02/prueba_entorno con Live Server; un commit de prueba y git push (la primera vez pide iniciar sesión en GitHub con el navegador).	15

Antes de la sesión

- Confirme con soporte del laboratorio si las computadoras ya tienen **VS Code** y **Git**. Git requiere permisos de administrador para instalarse: si falta, solicítelo con anticipación.
- Averigüe si los estudiantes entran con un **usuario de Windows propio** o compartido: de eso depende cómo se configura la identidad de Git.
- Pida en el canal que todos hayan hecho **git push** de la S02: lo que no esté en GitHub no aparecerá al clonar.

Punto de control: cada estudiante tiene su repositorio clonado en C:\INA de la computadora del laboratorio, la prueba del entorno funciona con Live Server y un commit de prueba llegó a GitHub.

3. Exposición (45 min) · notas del orador

Diapositiva 1 · Portada

Sesión 3, presencial en el laboratorio. Primero se repite la instalación y la configuración del entorno en las computadoras del laboratorio (sección 8 de la guía de instalación); después se escribe el primer script. Las diapositivas 2 a 7 son la exposición.

Diapositiva 2 · Tres formas de agregar JavaScript a una página

Recomendación del curso: archivos externos. Separan el comportamiento (JS) de la estructura (HTML), se reutilizan en varias páginas y el navegador los guarda en caché.

¿Dónde poner el script? Si va en el head SIN defer, se ejecuta antes de que exista el contenido de la página; en la UA2, cuando el código busque elementos del HTML, no los encontrará. Por eso: al final del body o en el head con defer.

La ruta del src es relativa al index.html: js/app.js significa «dentro de la carpeta js». Un error en la ruta no muestra nada en la página: solo un 404 en la consola (lo verán en la práctica guiada).

Elemento	Código	Explicación
Interno	`<script> console.log("Hola"); </script>`	Código dentro del HTML. Solo para pruebas.
Externo al final del body	`<script src="js/app.js"></script>`	Se ejecuta cuando la página ya se leyó.
Externo con defer	`<script src="js/app.js" defer></script>`	En el head: se ejecuta al terminar de cargar.
Archivo .js	`js/app.js`	Solo JavaScript: sin etiquetas <script>.
Ruta relativa	`src="js/app.js"`	Desde la carpeta del index.html.
Con src, nada adentro	`<script src="…">código</script>`	✗ El código interno se ignora.

Diapositiva 3 · Interno y externo en la misma página

Pida al grupo que prediga el orden antes de mostrar la salida. El navegador lee el HTML de arriba hacia abajo: encuentra el script externo en el head, pero defer le dice «descárgalo y ejecútalo cuando termines la página». Sigue leyendo, encuentra el script interno y lo ejecuta en ese momento. Al terminar la página, ejecuta app.js.

La segunda línea de app.js muestra document.title: la página ya cargó y JavaScript puede leer sus datos. Es un adelanto del DOM (sesión 10).

script · index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>La etiqueta script</title>
  <!-- Externo en el head: defer espera a que cargue la página -->
  <script src="js/app.js" defer></script>
</head>
<body>
  <h1>Mi primera página con JavaScript</h1>
  <!-- Interno: el código va dentro de la etiqueta -->
  <script>
    console.log("Interno: se ejecuta al leerlo");
  </script>
</body>
</html>
```

script · js/app.js

```
// app.js - Archivo externo (enlazado con src y defer)
console.log("Externo con defer: se ejecuta al final");
console.log("La página ya cargó:", document.title);
```

Consola del navegador (ejecución real en Chromium)

```
Interno: se ejecuta al leerlo
Externo con defer: se ejecuta al final
La página ya cargó: La etiqueta script
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
html 7	`<script src="js/app.js" defer></script>`	Enlaza el archivo externo.	src indica la ruta; defer pospone la ejecución hasta que la página termine de cargar.	Se ejecuta de último
html 12	`<script> … </script>`	Script interno.	El código está entre las etiquetas; se ejecuta en cuanto el navegador llega a esa línea.	Primer mensaje
js 2	`console.log("Externo con defer…");`	Mensaje del archivo externo.	console.log escribe en la consola del navegador.	Segundo mensaje
js 3	`console.log("…", document.title);`	Dos valores en un mensaje.	La coma separa los valores; document.title es el título de la pestaña.	La etiqueta script

Diapositiva 4 · Las reglas básicas de escritura

Comentarios: los mismos de C. Úselos para explicar el porqué, no para repetir el código.

Punto y coma: en JavaScript es opcional porque existe la inserción automática (ASI), pero hay casos raros en que falla. Convención del curso: siempre terminar las instrucciones con punto y coma, como en C.

JavaScript distingue mayúsculas y minúsculas: console.log funciona, console.Log o Console.log dan error.

Textos: comillas dobles o simples son equivalentes; las invertidas (plantillas) permiten varias líneas e insertar valores con ${…} (sesión 2). Convención del curso: comillas dobles.

Elemento	Código	Explicación
Comentario de línea	`// explica esta línea`	Igual que en C.
Comentario de bloque	`/* varias líneas */`	Igual que en C.
Punto y coma	`console.log("Hola");`	Termina cada instrucción.
Mayúsculas importan	`console.log ≠ console.Log`	Log con L mayúscula no existe.
Textos	`"doble" · 'simple' · `invertida``	Tres tipos de comillas.
Secuencias de escape	`\n  \t  \"  \'  \\  \u00F1`	Caracteres especiales.

Diapositiva 5 · Comentarios y secuencias de escape

Proyecte el archivo y pida al grupo que prediga cada línea antes de mostrar la consola. Las más interesantes: \\ (para mostrar UNA barra se escriben dos) y \u00F1, que escribe la ñ con su código Unicode; sirve para símbolos que no están en el teclado, como el del colón (\u20A1), que usarán en la práctica autónoma.

La última línea no tiene punto y coma y funciona: el navegador lo inserta automáticamente (ASI).

basica · js/basica.js

```
// basica.js - Comentarios, punto y coma y secuencias de escape
/* Comentario de varias líneas:
   el navegador no lo ejecuta. */
console.log("Hola, INA");              // termina con punto y coma
console.log("Línea 1\nLínea 2");       // \n  salto de línea
console.log("Código\tPrecio");         // \t  tabulación
console.log("Dijo: \"¡Listo!\"");      // \"  comilla doble
console.log('It\'s JavaScript');       // \'  comilla simple
console.log("Carpeta: C:\\INA");       // \\  barra invertida
console.log("Ma\u00F1ana");            // \u  carácter Unicode (ñ)
console.log("Sin punto y coma")        // también funciona (ASI)
```

Consola del navegador (ejecución real en Chromium)

```
Hola, INA
Línea 1
Línea 2
Código  Precio
Dijo: "¡Listo!"
It's JavaScript
Carpeta: C:\INA
Mañana
Sin punto y coma
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
2–3	`/* … */`	Comentario de bloque.	Todo lo que está entre /* y */ se ignora, aunque ocupe varias líneas.	No se ejecuta
4	`console.log("Hola, INA");`	Instrucción completa.	console.log muestra el texto; el ; termina la instrucción.	Hola, INA
5	`"Línea 1\nLínea 2"`	\n: salto de línea.	Divide el texto en dos líneas.	Dos líneas
6	`"Código\tPrecio"`	\t: tabulación.	Salta a la siguiente columna (múltiplo de 8).	Código Precio
7	`"Dijo: \"¡Listo!\""`	\": comilla doble.	Sin la barra, la comilla terminaría el texto.	Dijo: "¡Listo!"
8	`'It\'s JavaScript'`	\': comilla simple.	Igual que la anterior, para textos entre comillas simples.	It's JavaScript
9	`"Carpeta: C:\\INA"`	\\: barra invertida.	Una sola barra iniciaría una secuencia; dos barras muestran una.	C:\INA
10	`"Ma\u00F1ana"`	\u: carácter Unicode.	\u seguido de 4 dígitos hexadecimales: 00F1 es la ñ.	Mañana
11	`console.log("Sin punto y coma")`	Sin ;.	El navegador inserta el ; automáticamente (ASI).	Funciona

Diapositiva 6 · La consola: la ventana del programador

Abra la consola en vivo (F12, pestaña Console). Muestre que se pueden escribir expresiones directamente: 2 + 3, "Hola".length. Es la calculadora del programador.

console.warn y console.error se ven en amarillo y rojo, y se pueden filtrar con los niveles (Default levels). Muestre el enlace archivo:línea de un error: al hacer clic, la consola abre el archivo en esa línea (pestaña Sources).

La consola es solo para la persona programadora: el usuario final no la ve. Para mostrar resultados en la página se usará el DOM (sesión 10).

consola · js/consola.js

```
1  // consola.js - Los métodos más usados de la consola
2  console.log("Mensaje normal");
3  console.info("Información para la persona programadora");
4  console.warn("Advertencia: el precio está vacío");
5  console.error("Error: no se encontró el producto");
6  console.log("Total:", 1500 + 900);   // varios valores, separados por coma
```

Consola del navegador (ejecución real en Chromium)

```
Mensaje normal
Información para la persona programadora
⚠ Advertencia: el precio está vacío
✕ Error: no se encontró el producto
Total: 2400
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
2	`console.log("Mensaje normal");`	Mensaje común.	Se usa para mostrar resultados y revisar el programa.	Texto normal
3	`console.info("…");`	Información.	En Chrome se ve igual que log; algunos navegadores le agregan un ícono.	Texto normal
4	`console.warn("…");`	Advertencia.	Fondo amarillo y un ícono ⚠.	⚠ Advertencia…
5	`console.error("…");`	Error.	Fondo rojo y un ícono ✕; el programa sigue ejecutándose.	✕ Error…
6	`console.log("Total:", 1500 + 900);`	Varios valores.	La coma separa los valores; primero se calcula 1500 + 900.	Total: 2400

Diapositiva 7 · Lo esencial de hoy

Cierre de la exposición. Repase la tabla en voz alta pidiendo a distintas personas que completen la idea clave antes de mostrarla.

Los tres errores frecuentes se provocan a propósito en la práctica guiada: así los reconocerán cuando les ocurran.

Tema	Idea clave
<script>	Externo: al final del body o en el head con defer
Archivo .js	Solo JavaScript; ruta relativa desde el index.html
Sintaxis básica	// y /* */ · ; al final · mayúsculas importan · comillas dobles
Escape	\n \t \" \' \\ \u00F1
Consola	F12 · log, warn, error · tipo, mensaje y archivo:línea

- **Error frecuente:** Ruta del src incorrecta: la consola muestra 404.

- **Error frecuente:** console.Log con L mayúscula: TypeError.
- **Error frecuente:** Comillas sin cerrar o copiadas de Word (“ ”): SyntaxError.

4. Práctica guiada (50 min)

Momento	Min	Qué hace el docente	Punto de control
Ejercicio · Ficha del estudiante → UA1/S03/ficha/	30	Proyecto con index.html y js/ficha.js; comentarios, \t, \", \\, \n y console.warn.	La ficha de 9 líneas y el aviso aparecen en la consola.
Demostración · Errores frecuentes	20	Ruta del src, console.Log, comillas sin cerrar y comillas de Word.	El grupo lee el mensaje y la línea antes de corregir.

Convención del módulo para el código

Cada archivo empieza con el **enunciado**, **qué se logra**, **cómo se logra** y el **resultado esperado**. Antes de cada línea va un comentario con **QUÉ** hace, **CÓMO** lo hace y **PARA QUÉ** se usa. Los textos pueden llevar tildes y ñ (archivos en UTF-8). Todas las instrucciones terminan en **;** y los textos usan **comillas dobles**.

4.1 Ejercicio · Ficha del estudiante → UA1/S03/ficha/ (30 min)

Qué se logra	Crear un proyecto con un archivo .js externo y usar comentarios, punto y coma y secuencias de escape.
Qué es nuevo	Estructura index.html + js/; <script src> al final del body; \t, \", \\ y \n; console.warn.
Resultado esperado	La ficha de 9 líneas en la consola y un aviso amarillo.

Comando (terminal de VS Code · Git Bash)	¿Qué hace?	Resultado
`cd /c/INA/CSTI12004-Nombre-Apellido`	Entra al repositorio.	(main)
`mkdir -p UA1/S03/ficha/js`	Crea la carpeta del proyecto y su subcarpeta js.	Sin mensajes

Paso 1. La página. Cree index.html con ! + Tab (Emmet) y ajústelo con el grupo. Pregunte: ¿por qué el script va al final del body? (Para que se ejecute cuando la página ya se leyó).

ficha/index.html

```
<!DOCTYPE html>
<!-- index.html - Página de la ficha del estudiante (sesión 1)
     QUÉ SE LOGRA: enlazar un archivo JavaScript externo a una página.
     CÓMO SE LOGRA: la etiqueta script con el atributo src al final del body.
     RESULTADO ESPERADO: la página muestra el título y la consola (F12)
     muestra la ficha. -->
<html lang="es">
<head>
  <!-- QUÉ: codificación de caracteres. CÓMO: meta charset.
       PARA QUÉ: que las tildes y la ñ se vean bien. -->
  <meta charset="UTF-8">
  <!-- QUÉ: ancho de la página en celulares. CÓMO: meta viewport.
```

```
        PARA QUÉ: que la página se adapte a la pantalla. -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- QUÉ: título de la pestaña. CÓMO: etiqueta title.
        PARA QUÉ: identificar la página en el navegador. -->
  <title>Ficha del estudiante</title>
</head>
<body>
  <!-- QUÉ: contenido visible. CÓMO: un encabezado y un párrafo.
        PARA QUÉ: indicar dónde ver el resultado. -->
  <h1>Ficha del estudiante</h1>
  <p>Abra la consola (F12) para ver la ficha.</p>
  <!-- QUÉ: enlace al archivo JavaScript.
        CÓMO: script con src y la ruta RELATIVA desde este archivo.
        PARA QUÉ: separar el código del HTML; al ir al final del body,
        el código se ejecuta cuando la página ya se leyó. -->
  <script src="js/ficha.js"></script>
</body>
</html>
```

Paso 2. El script. Escriba primero el encabezado de comentarios y las tres líneas del marco; abra con Live Server y la consola (F12) para comprobar el enlace. Luego agregue los datos uno a uno, prediciendo con el grupo qué mostrará cada secuencia de escape.

ficha/js/ficha.js

```
// ficha.js - Ficha del estudiante en la consola del navegador
// ENUNCIADO: mostrar en la consola una ficha con un título enmarcado, el
//   nombre, el módulo, un lema entre comillas y la ruta del repositorio,
//   con los datos alineados, y un aviso al final.
// QUÉ SE LOGRA: escribir el primer archivo .js enlazado a una página y
//   usar comentarios, punto y coma y secuencias de escape.
// CÓMO SE LOGRA: una instrucción console.log por línea de la ficha; \t
//   alinea los datos, \" muestra comillas, \\ una barra invertida y \n
//   un salto de línea; console.warn muestra el aviso.
// RESULTADO ESPERADO: la ficha de 9 líneas en la consola (F12) y un aviso
//   amarillo al final.
// CÓMO LEER LOS COMENTARIOS: antes de cada línea se explica QUÉ hace,
//   CÓMO lo hace y PARA QUÉ se usa.

/* Este es un comentario de varias líneas: el navegador ignora todo lo
   que está entre la barra-asterisco y el asterisco-barra. Sirve para
   explicaciones largas o para desactivar un bloque de código. */

// QUÉ: línea superior del marco.
// CÓMO: console.log muestra en la consola el texto entre comillas.
// PARA QUÉ: separar la ficha de otros mensajes de la consola.
console.log("========================================");
// QUÉ: título de la ficha.
// CÓMO: los espacios dentro de las comillas también se muestran.
// PARA QUÉ: centrar el título dentro del marco.
console.log("   FICHA DEL ESTUDIANTE · CSTI12004");
// QUÉ: línea inferior del marco. CÓMO: igual que la primera.
// PARA QUÉ: cerrar el título.
console.log("========================================");
// QUÉ: nombre del estudiante.
// CÓMO: \t inserta una tabulación entre la etiqueta y el dato.
// PARA QUÉ: que todos los datos empiecen en la misma columna.
console.log("Nombre:\tMaría Núñez");
// QUÉ: módulo que cursa. CÓMO: otra tabulación con \t.
// PARA QUÉ: alinear el dato con el nombre.
console.log("Módulo:\tProgramación web en el entorno cliente");
// QUÉ: lema entre comillas.
// CÓMO: \" pone una comilla doble DENTRO de un texto que ya está entre
```

```
//   comillas dobles; sin la barra invertida, el texto terminaría ahí.
// PARA QUÉ: citar una frase.
console.log("Lema:\t\"Aprendo haciendo\"");
// QUÉ: ruta del repositorio en Windows.
// CÓMO: \\ muestra UNA barra invertida; una sola barra iniciaría una
//   secuencia de escape.
// PARA QUÉ: escribir rutas de carpetas de Windows.
console.log("Repo:\tC:\\INA\\CSTI12004-Maria-Nunez");
// QUÉ: cierre del marco y una línea final, en una sola instrucción.
// CÓMO: \n hace un salto de línea dentro del texto.
// PARA QUÉ: mostrar dos líneas con un solo console.log.
console.log("========================================\nFin de la ficha");
// QUÉ: aviso final.
// CÓMO: console.warn muestra el mensaje con fondo amarillo y un ícono.
// PARA QUÉ: distinguir los avisos de los mensajes normales.
console.warn("Recuerde hacer commit y push de la S03");
```

Consola del navegador (ejecución real en Chromium)

```
========================================
   FICHA DEL ESTUDIANTE · CSTI12004
========================================
Nombre: María Núñez
Módulo: Programación web en el entorno cliente
Lema:   "Aprendo haciendo"
Repo:   C:\INA\CSTI12004-Maria-Nunez
========================================
Fin de la ficha
⚠ Recuerde hacer commit y push de la S03
```

Pida que cambien los datos por los suyos. Pregunte: ¿qué pasa si quitan la barra antes de la comilla en la línea del lema? (El texto termina antes y aparece un SyntaxError: ninguna línea se muestra).

Punto de control: todas las personas ven la ficha con los datos alineados y el aviso amarillo, sin errores rojos.

Commit: git add UA1/S03/ficha · git commit -m "S03: agrega ficha del estudiante" · git push

4.2 Demostración · Errores frecuentes (20 min)

Provoque cada error en la ficha, en vivo, y pida al grupo que lo provoque también en su copia. Antes de corregir, pida que lean en voz alta **el tipo de error, el mensaje y el archivo:línea**.

Error 1 · Ruta del src

errores/1_ruta · index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Error 1 · Ruta del archivo</title>
</head>
<body>
  <h1>Error 1 · Ruta del archivo</h1>
  <!-- ERROR: el archivo se llama ficha.js, no fichas.js -->
  <script src="js/fichas.js"></script>
</body>
</html>
```

Consola del navegador (ejecución real en Chromium)

```
✕ GET http://127.0.0.1:5500/js/fichas.js net::ERR_ABORTED 404 (Not Found)
```

La página se ve bien, pero el script no se cargó: el archivo se llama ficha.js y el src dice fichas.js. 404 significa «no encontrado». **Corrección:** revisar el nombre y la carpeta del src (mayúsculas incluidas: en un servidor Linux Ficha.js y ficha.js son archivos distintos).

Error 2 · Mayúsculas

errores/2_mayuscula · js/app.js

```
// Error 2 · JavaScript distingue mayúsculas de minúsculas
console.log("Primera línea: funciona");
console.Log("Segunda línea: Log con L mayúscula");
console.log("Tercera línea: ¿se ejecuta?");
```

Consola del navegador (ejecución real en Chromium)

```
Primera línea: funciona
✕ Uncaught TypeError: console.Log is not a function   app.js:3
```

La línea 2 se ejecuta; en la 3, console.Log no existe y el programa **se detiene**: la línea 4 no se muestra. TypeError: se usó como función algo que no lo es. **Corrección:** console.log en minúscula.

Error 3 · Comillas sin cerrar

errores/3_comillas · js/app.js

```
// Error 3 · Comillas sin cerrar
console.log("Primera línea: ¿se ejecuta?");
console.log("Segunda línea sin cerrar);
```

Consola del navegador (ejecución real en Chromium)

```
✕ Uncaught SyntaxError: Invalid or unexpected token   app.js:3
```

Error de **sintaxis**: no se ejecuta **ninguna** línea, ni la 2. Compárelo con el error 2, que sí dejó ejecutar las líneas anteriores. **Corrección:** cerrar la comilla.

Error 4 · Comillas de Word

errores/4_tipograficas · js/app.js

```
1  // Error 4 · Comillas tipográficas copiadas de Word o de un PDF
2  console.log(“Hola desde Word”);
```

Consola del navegador (ejecución real en Chromium)

```
✕ Uncaught SyntaxError: Invalid or unexpected token   app.js:2
```

Las comillas “ ” (tipográficas) no son comillas de JavaScript. Ocurre al copiar código desde Word, un PDF o un chat. **Corrección:** reescribir las comillas en VS Code.

> **Idea para cerrar la demostración**
>
> Hay errores que **impiden ejecutar todo el archivo** (sintaxis: SyntaxError) y errores que **detienen el programa en una línea** (ejecución: TypeError, ReferenceError). Un 404 no es un error de JavaScript: el archivo ni siquiera llegó al navegador.

5. Práctica autónoma (45 min) · soluciones

El enunciado está en **S03_Practica_autonoma_Estudiantes**. Se hace en clase: acompañe sin dar la solución y use las tablas «Qué revisar».

5.1 Solución · Menú de la soda → UA1/S03/soda/

soda/index.html

```
1  <!DOCTYPE html>
2  <!-- index.html - Página del menú de la soda (práctica autónoma S03)
3       QUÉ SE LOGRA: enlazar el archivo js/menu.js a la página.
4       RESULTADO ESPERADO: la página muestra el nombre de la soda y la
5       consola (F12) muestra el menú completo. -->
6  <html lang="es">
7  <head>
8    <!-- QUÉ: codificación UTF-8. PARA QUÉ: tildes, ñ y el símbolo ¢. -->
9    <meta charset="UTF-8">
10   <meta name="viewport" content="width=device-width, initial-scale=1.0">
11   <title>Soda La Esquina</title>
12   <!-- QUÉ: enlace al archivo del menú.
13        CÓMO: script con src y defer en el head.
14        PARA QUÉ: defer descarga el archivo mientras se lee la página y lo
15        ejecuta cuando la página terminó de cargar. -->
16   <script src="js/menu.js" defer></script>
17 </head>
18 <body>
19   <h1>Soda La Esquina</h1>
20   <p>Abra la consola (F12) para ver el menú del día.</p>
21 </body>
22 </html>
```

soda/js/menu.js

```
1  // menu.js - Menú del día de la Soda La Esquina en la consola
2  // ENUNCIADO: mostrar en la consola el menú del día de una soda: título
3  //   enmarcado, tres secciones (desayunos, almuerzos y bebidas) con precio
```

```
//   y nombre de cada plato, una invitación con una frase entre comillas,
//   la carpeta donde se guarda el proyecto, un mensaje de despedida en dos
//   líneas y un aviso.
// QUÉ SE LOGRA: practicar comentarios, punto y coma y todas las
//   secuencias de escape vistas en la sesión.
// CÓMO SE LOGRA: un console.log por línea; \t alinea los precios, \u20A1
//   escribe el símbolo del colón, \" las comillas, \\ las barras de la
//   ruta y \n el salto de línea; console.warn muestra el aviso.
// RESULTADO ESPERADO: 17 líneas de menú en la consola y un aviso amarillo.
// CÓMO LEER LOS COMENTARIOS: antes de cada línea se explica QUÉ hace,
//   CÓMO lo hace y PARA QUÉ se usa.

/* Estructura del menú:
   1. Título enmarcado.
   2. Secciones con sus platos (precio y nombre).
   3. Cierre: invitación, carpeta, despedida y aviso. */

// QUÉ: marco superior. CÓMO: console.log con un texto de signos =.
// PARA QUÉ: separar el menú de otros mensajes.
console.log("==========================================");
// QUÉ: nombre de la soda y título.
// CÓMO: espacios al inicio para centrarlo; el · se escribe directo.
// PARA QUÉ: identificar el menú.
console.log("     SODA LA ESQUINA · MENÚ DEL DÍA");
// QUÉ: marco inferior del título. CÓMO: igual que el superior.
// PARA QUÉ: cerrar el título.
console.log("==========================================");

// QUÉ: título de la primera sección. CÓMO: texto en mayúsculas.
// PARA QUÉ: agrupar los platos del desayuno.
console.log("DESAYUNOS");
// QUÉ: primer plato.
// CÓMO: \t sangra la línea, \u20A1 escribe el símbolo ₡ y el segundo \t
//   separa el precio del nombre.
// PARA QUÉ: que precios y nombres queden en columnas.
console.log("\t\u20A12 500\tGallo pinto con huevo");
// QUÉ: segundo plato. CÓMO: la misma estructura que el anterior.
// PARA QUÉ: mantener las columnas alineadas.
console.log("\t\u20A11 800\tEmpanada de queso y café");

// QUÉ: segunda sección y sus platos. CÓMO: igual que la primera.
// PARA QUÉ: agrupar los almuerzos.
console.log("ALMUERZOS");
console.log("\t\u20A13 900\tCasado con pollo");
console.log("\t\u20A14 200\tCasado con pescado");

// QUÉ: tercera sección y sus bebidas. CÓMO: igual que las anteriores.
// PARA QUÉ: agrupar las bebidas.
console.log("BEBIDAS");
console.log("\t\u20A11 000\tFresco natural");
console.log("\t\u20A1  900\tCafé chorreado");

// QUÉ: línea divisoria. CÓMO: texto de guiones.
// PARA QUÉ: separar el menú del cierre.
console.log("------------------------------------------");
// QUÉ: invitación con una frase entre comillas.
// CÓMO: \" escribe comillas dobles dentro del texto.
// PARA QUÉ: destacar el nombre del plato especial.
console.log("Pregunte por el \"plato del día\".");
// QUÉ: carpeta del proyecto en Windows.
// CÓMO: cada \\ muestra una barra invertida.
// PARA QUÉ: practicar rutas de Windows dentro de un texto.
console.log("Proyecto: C:\\INA\\CSTI12004\\UA1\\S03\\soda");
// QUÉ: despedida en dos líneas con una sola instrucción.
// CÓMO: \n hace el salto de línea.
```

```
// PARA QUÉ: escribir dos líneas con un solo console.log.
console.log("¡Gracias por su visita!\nVuelva pronto.");
// QUÉ: aviso de un plato agotado.
// CÓMO: console.warn lo muestra con fondo amarillo.
// PARA QUÉ: llamar la atención del lector de la consola.
console.warn("Hoy no hay pescado: consulte el plato del día");
```

Consola del navegador (ejecución real en Chromium)

```
==========================================
     SODA LA ESQUINA · MENÚ DEL DÍA
==========================================
DESAYUNOS
        ₡2 500  Gallo pinto con huevo
        ₡1 800  Empanada de queso y café
ALMUERZOS
        ₡3 900  Casado con pollo
        ₡4 200  Casado con pescado
BEBIDAS
        ₡1 000  Fresco natural
        ₡  900  Café chorreado
------------------------------------------
Pregunte por el "plato del día".
Proyecto: C:\INA\CSTI12004\UA1\S03\soda
¡Gracias por su visita!
Vuelva pronto.
⚠ Hoy no hay pescado: consulte el plato del día
```

Qué revisar	Por qué
`\u20A1` para el símbolo ₡	Es el requisito de la secuencia \u; si escribieron ₡ directamente funciona, pero no cumple el requisito.
Las columnas de precios alineadas	Comprueba el uso de \t; si usaron espacios, se ve igual pero no practicaron la secuencia.
`\"` en «plato del día» y `\\` en la ruta	Las dos secuencias que más se olvidan.
Un solo console.log con \n en la despedida	Es requisito: dos líneas con una instrucción.
Encabezado de comentarios y comentario de bloque	Convención del módulo; el comentario de bloque es requisito.
Enlace con defer en el head	Requisito del enunciado: practicar la tercera forma de enlazar.

5.2 Solución · Horario con errores → UA1/S03/horario/

Se corrigen **uno por uno**, leyendo la consola después de cada corrección. Estas son las cuatro ejecuciones reales:

Ejecución 1 · Archivos tal como se entregan

```
✕ GET http://127.0.0.1:5500/js/horario.js net::ERR_ABORTED 404 (Not Found)
```

404: el src pide horario.js y el archivo es horarios.js.

Ejecución 2 · Después de corregir la ruta

```
✕ Uncaught SyntaxError: Invalid or unexpected token   horarios.js:3
```

SyntaxError en la línea 3: faltan las comillas de cierre. No se ejecuta nada.

Ejecución 3 · Después de cerrar las comillas

```
HORARIO DE ATENCIÓN
Lun a vie:      6:00 a. m. a 5:00 p. m.
✕ Uncaught ReferenceError: Console is not defined   horarios.js:4
```

Se ejecutan las líneas 2 y 3; la 4 falla: Console (con C mayúscula) no existe.

Ejecución 4 · Después de corregir Console

```
HORARIO DE ATENCIÓN
Lun a vie:      6:00 a. m. a 5:00 p. m.
Sábados:        7:00 a. m. a 12:00 m. d.
Domingos:       cerrado
```

Horario completo. La línea 5 no tenía punto y coma: funcionaba (ASI), pero por convención se agrega.

horario/index.html (corregido)

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="UTF-8">
5    <title>Horario de la soda</title>
6  </head>
7  <body>
8    <h1>Horario de la soda</h1>
9    <p>Abra la consola (F12) para ver el horario.</p>
10   <!-- CORRECCIÓN 1: el archivo se llama horarios.js (con s) -->
11   <script src="js/horarios.js"></script>
12 </body>
13 </html>
```

horario/js/horarios.js (corregido)

```
1  // horarios.js - Horario de atención de la soda (corregido)
2  console.log("HORARIO DE ATENCIÓN");
3  // CORRECCIÓN 2: faltaba cerrar las comillas antes del paréntesis.
4  console.log("Lun a vie:\t6:00 a. m. a 5:00 p. m.");
5  // CORRECCIÓN 3: console va en minúscula (Console no existe).
6  console.log("Sábados:\t7:00 a. m. a 12:00 m. d.");
7  // ESTILO: se agrega el punto y coma que faltaba (no era un error).
8  console.log("Domingos:\tcerrado");
```

#	Mensaje de la consola	Archivo:línea	Corrección
1	GET …/js/horario.js net::ERR_ABORTED 404 (Not Found)	index.html (src)	src="js/horarios.js"
2	Uncaught SyntaxError: Invalid or unexpected token	horarios.js:3	Cerrar las comillas: …5:00 p. m.");
3	Uncaught ReferenceError: Console is not defined	horarios.js:4	console en minúscula
4	(sin mensaje)	horarios.js:5	Agregar ; por convención

5.3 Reto · Clasificación MVC

Elemento	Componente
Los nombres y precios de los platos guardados en el programa.	Modelo
La página HTML con el menú y sus estilos CSS.	Vista
El código que, al presionar «Agregar», toma el plato elegido y pide actualizar el pedido.	Controlador
El total del pedido y la lista de platos pedidos.	Modelo
El mensaje «Pedido enviado» que aparece en la pantalla.	Vista
El código que, al presionar «Enviar», revisa que haya al menos un plato y, si no, pide a la vista mostrar un aviso.	Controlador

Acepte justificaciones razonables: lo importante es que distingan **datos** (modelo), **pantalla** (vista) y **coordinación** (controlador).

6. Síntesis (10 min)

¿Qué muestra la consola?

Proyecte el código y pida que escriban la salida en el cuaderno antes de ejecutarlo:

sintesis · js/app.js

```
1  // app.js - ¿Qué muestra la consola?
2  console.log("A\tB\nC\\D");
3  console.log("Ella dijo \"hola\"");
4  console.warn("Fin")
```

Consola del navegador (ejecución real en Chromium)

```
A       B
C\D
Ella dijo "hola"
⚠ Fin
```

Preguntas rápidas

Pregunta	Respuesta
¿Qué diferencia hay entre un error de sintaxis y uno de semántica?	El de sintaxis impide ejecutar el archivo; el de semántica se ejecuta pero da un resultado incorrecto.
¿Dónde se coloca la etiqueta script de un archivo externo?	Al final del body o en el head con defer.
¿Cómo se muestra una barra invertida dentro de un texto?	Con \\ (dos barras).
¿Qué significa un 404 en la consola?	El navegador no encontró el archivo: la ruta o el nombre del src están mal.

7. Próxima sesión (5 min)

Tema: Sesión 4 · viernes 13 de noviembre de 2026 (virtual) · Variables, constantes, tipos de datos y operadores; solicitud de valores por pantalla.

Pregunta para pensar: En C escribía int edad = 20; y const double IVA = 0.13;. En JavaScript no se escribe el tipo: ¿cómo cree que sabe el navegador si un valor es un número o un texto?

8. Cierre (5 min)

- Cada estudiante hace **git push** y verifica en GitHub que se ven UA1/S03/ficha, soda y horario.
- Revise al azar dos repositorios en el proyector: README de la sesión, capturas y commits con mensajes «S03: …».
- **Cierre de la sesión de GitHub** en cada computadora del laboratorio: Administrador de credenciales → Credenciales de Windows → git:https://github.com → Quitar.
- Recuerde: en la casa, antes de trabajar, **git pull**.